COMPUTER SCIENCE AND ENGINEERING

INTERNET OF THINGS (IoT)

2025 - 2026

Challenge #1
Wokwi and Power Consumption

Team:

Elena Lin (Leader) 10802468

Jiayi Zheng 10811585

Wokwi public link:

IoTChallenge.ino Copy - Wokwi ESP32, STM32, Arduino Simulator

Contents

1| System Implementation

1.1 System objective

This challenge aims to develop a Wokwi-based IoT motion sensor node capable of detecting human motion and ambient light, and transmitting this information to a sink node for lighting control.

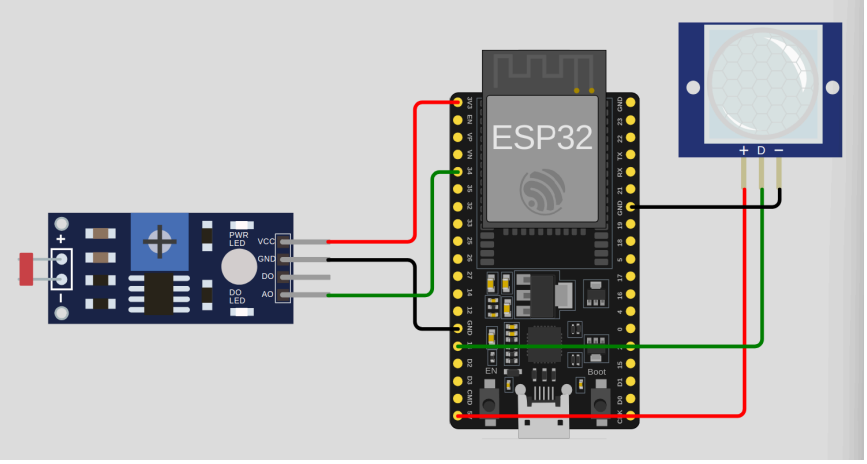
1.2 Hardware and Wokwi setup

The system is implemented in Wokwi using an ESP32 board, a PIR motion sensor, and an LDR sensor.

The ESP32 collects the sensor data and transmits it to the sink node via ESP-NOW.

The PIR sensor is used for motion detection. Its + pin connected to 5V, its − pin to GND, and its digital output *D* to GPIO 13, defined in the code as *PIR_PIN*.

The LDR sensor is used to measure ambient light. Its *VCC* pin is connected to 3.3V, its *GND* pin to GND, and its analog output *AO* to GPIO 34, defined as *LDR_PIN*.



1.3 Code Logic

This section presents the main software logic of the sensor node. To keep the code clear and concise, the implementation is divided into separate functions, while *setup()* contains the main workflow executed after each wake-up.

1.3.1 Library Inclusion and System Configuration

At the beginning of the code, the required libraries, pin definitions, and main system parameters are declared.

WiFi.h is included to configure the ESP32 wireless interface, *esp_now.h* enables ESP-NOW communication, and *esp_sleep.h* provides the functions required for Deep Sleep operation.

It also defines the sensor pins: *PIR_PIN* for motion detection and *LDR_PIN* for ambient light measurement.

The Deep Sleep interval is calculated according to the assignment requirement using the leader's person code:

$X[s] = \frac{(68\ \%\ 50)+5}{10} = 2.3s\ [10802468 \rightarrow \text{AB} = 68]$

which corresponds to 2,300,000 microseconds.

For wireless data transmission, the broadcast MAC address $FF{:}FF{:}FF{:}FF{:}FF{:}FF$ is used as the destination address.

In addition, a peer information structure is declared to store the parameters required for ESP-NOW communication.

```
#include <WiFi.h>
#include <esp_now.h>
#include <esp_sleep.h>

#define PIR_PIN 13    // PIR motion sensor pin
#define LDR_PIN 34    // LDR analog input pin

// Person code: 10802468
// Deep sleep time X[s] = [(68 mod 50) + 5] / 10 = 2.3s = 2300000 us
const uint64_t DEEP_SLEEP_US = 2300000ULL;

// Broadcast MAC address used as sink address
uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

esp_now_peer_info_t peerInfo;
```

1.3.2 setup()

The *setup ()* function implements one complete operating cycle of the IoT sensor node: sensing the environment, preparing the data packet, activating wireless communication, transmitting the packet with bounded waiting for completion, and then returning to deep sleep.

After wake-up, the node reads the motion state from the PIR sensor and the ambient light level from the LDR.

Once the sensor values are acquired, the node constructs the payload string to be transmitted. The message format follows the required format: $MOTION\_DETECTED - LUMINOSITY{:}ZZZ$

or $MOTION\_NOT\_DETECTED - LUMINOSITY{:}ZZZ$.

The communication subsystem is then initialized through *initEspNow ()*. After that, the message is transmitted using *esp_now_send ()*.

Since the send function only starts the transmission process, the program waits for the send callback to confirm completion before continuing.

After transmission, *printDebugInfo ()* is called to output the payload and measured timing values.

Finally, the node enters Deep Sleep through *goToDeepSleep ()*.

```
void setup() {
  Serial.begin(115200);
  pinMode(PIR_PIN, INPUT);

  unsigned long wakeUp = micros();

  // Measure sensor reading time
  unsigned long sensorReadStart = micros();
  int motion = digitalRead(PIR_PIN);   // read motion state from PIR
  int lightValue = analogRead(LDR_PIN); // read ambient light from LDR
  unsigned long sensorReadEnd = micros();

  String message;

  // Build message in required format
  unsigned long msgBuildStart = micros();
  if (motion == HIGH) {
    message = "MOTION_DETECTED-LUMINOSITY:" + String(lightValue);
  } else {
    message = "MOTION_NOT_DETECTED-LUMINOSITY:" + String(lightValue);
  }
  unsigned long msgBuildEnd = micros();

  // Initialize ESP-NOW
  unsigned long wifiOnStart = micros();
  if (!initEspNow()) {
    Serial.println("ESP-NOW init failed");
    return;
  }
  unsigned long wifiOnEnd = micros();

  // Send message via ESP-NOW
  unsigned long txStart = micros();
  esp_now_send(broadcastAddress, (uint8_t*)message.c_str(), message.length() + 1);

  // Wait until sending is complete or timeout after 500 ms
  unsigned long waitStart = millis();
  while (!sendDone && (millis() - waitStart < 500)) {
    delay(1);
  }

  unsigned long beforeSleep = micros();

  // Print measured values
  printDebugInfo(message, wakeUp, sensorReadStart, sensorReadEnd,
  msgBuildStart, msgBuildEnd, wifiOnStart, wifiOnEnd,
  txStart, beforeSleep);

  // Enter deep sleep
  goToDeepSleep();
}
```

1.3.3 OnDataSent()

OnDataSent () is the ESP-NOW send callback function. It is triggered when the transmission process finishes, and it stores the transmission end time, the send status, and the completion flag used by the main program.

```
17  // Variables used to measure transmission time and status
18  unsigned long txEnd = 0;
19  bool sendDone = false;
20  esp_now_send_status_t sendStatus;
21
22  void OnDataSent(const wifi_tx_info_t *mac_addr, esp_now_send_status_t status) {
23    txEnd = micros();          // store transmission end time
24    sendDone = true;           // mark transmission as completed
25    sendStatus = status;       // save transmission result
26  }
```

1.3.4 initEspNow()

This function initializes ESP-NOW, sets the ESP32 to station mode, registers the send callback, and adds the broadcast peer.

```
28  // Initialize ESP-NOW and add broadcast peer
29  bool initEspNow() {
30    WiFi.mode(WIFI_STA);
31
32    esp_now_init();
33    // Send callback
34    esp_now_register_send_cb(OnDataSent);
35
36    // Peer registration
37    memcpy(peerInfo.peer_addr, broadcastAddress, 6);
38    peerInfo.channel = 0;
39    peerInfo.encrypt = false;
40    // Add peer
41    esp_now_add_peer(&peerInfo);
42
43    return true;
44  }
```

1.3.5 goToDeepSleep()

This function enables timer-based wake-up and places the ESP32 into Deep Sleep in order to reduce energy consumption between sensing cycles.

```
91  // Enter deep sleep and wake up again after 2.3 seconds
92  void goToDeepSleep(){
93    Serial.flush();
94    esp_sleep_enable_timer_wakeup(DEEP_SLEEP_US);
95    esp_deep_sleep_start();
96  }
```

1.3.6 loop()

This function is empty because the complete logic is executed in *setup ()* after each wake up.

```
149   void loop() {
150     // Empty because all operations are done in setup() after each wake-up
151   }
```

1.4 Serial Output Examples

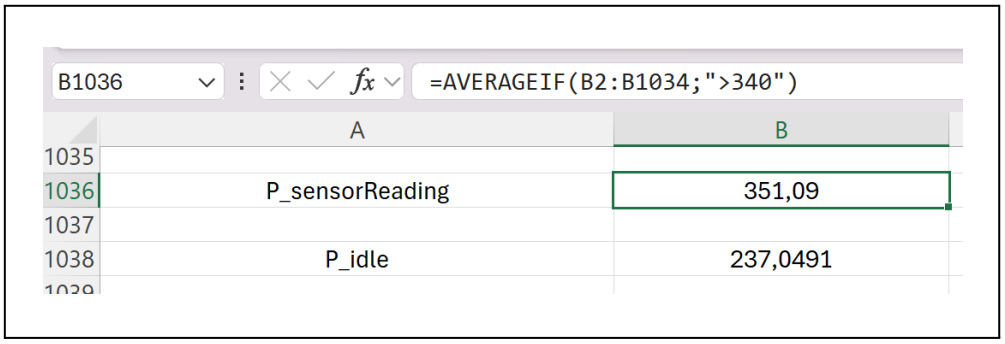
The serial output confirms that the node successfully wakes up from Deep Sleep, reads the motion and light sensors, formats the message correctly according to the sensing result, transmits it via ESP-NOW, and then returns to Deep Sleep.

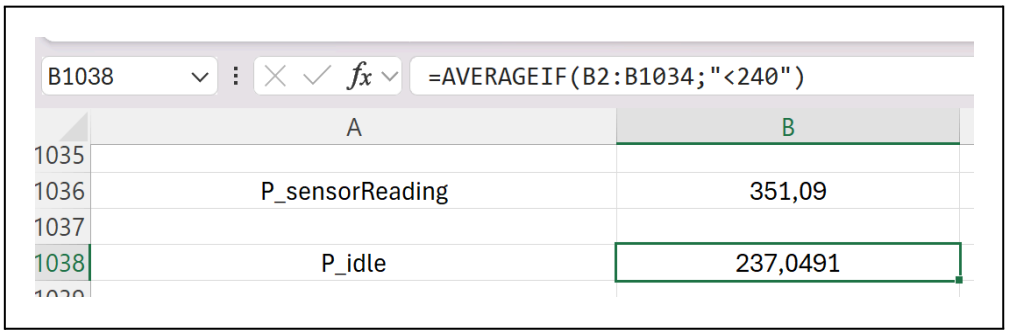
```
----- WAKE UP -----
Message: MOTION_DETECTED-LUMINOSITY:315
Sensor reading
Message building
Message transmitting
Send Status: Ok
----- GOING TO DEEP SLEEP -----
```

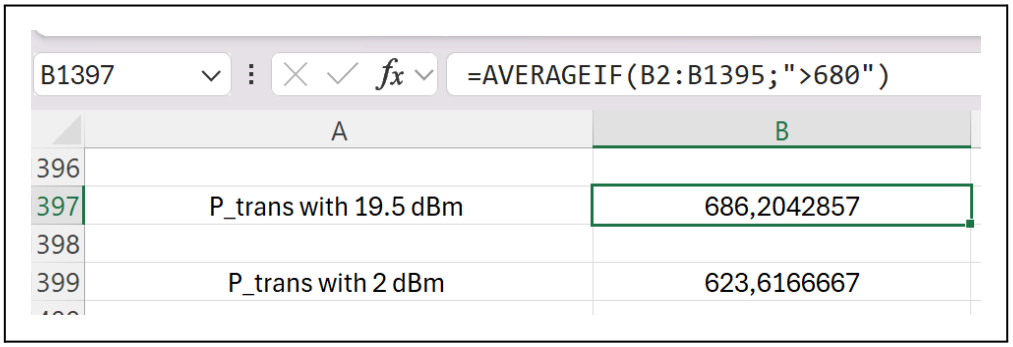
```
----- WAKE UP -----
Message: MOTION_NOT_DETECTED-LUMINOSITY:2242
Sensor reading
Message building
Message transmitting
Send Status: Ok
----- GOING TO DEEP SLEEP -----
```

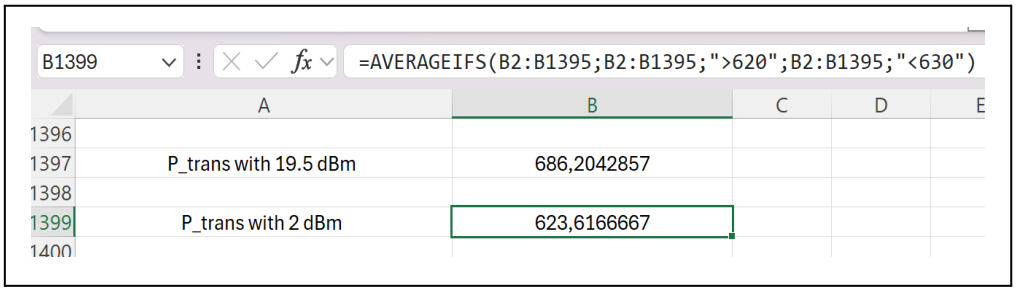
2| Energy Consumption Estimation

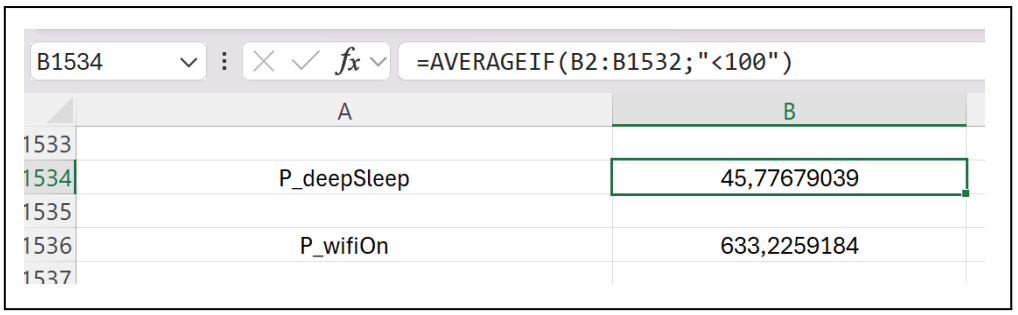
2.1 Power Consumption

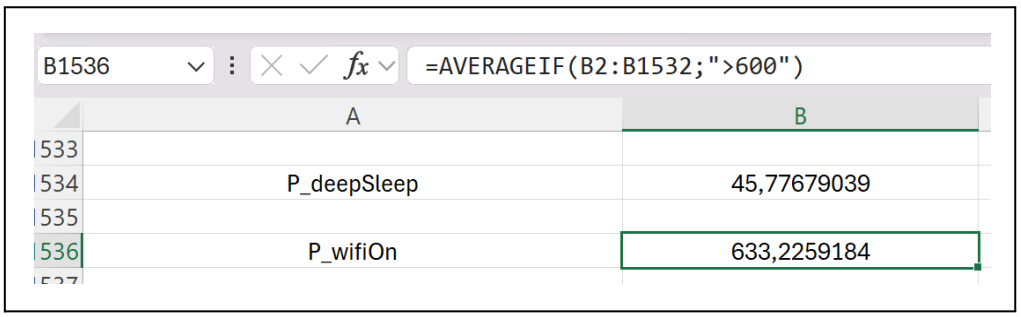












We calculate the average power consumption for the following states:

STATES	POWER (average)
Transmission at 19,5 dBm (>680 mW)	P_{trans} (19,5 dBm) ≈ 686,2 mW
Transmission at 2 dBm (620 - 630 mW)	P_{trans} (2 dBm) ≈ 623,62 mW
Sensor Reading (>340 mW)	P_{read} ≈ 351,09 mW
Idle (240 mW)	P_{idle} ≈ 237,05 mW
Deep Sleep (<100 mW)	P_{ds} ≈ 45,78 mW
WiFi on (>600 mW)	P_{w} ≈ 633,23 mW

The average power for each state was computed using the AVERAGEIF/AVERAGEIFS function in Excel, filtering the samples according to the corresponding power ranges provided in the CSV files.
For instance, transmission at 19,5 dBm was identified by selecting samples above 680 mW, while transmission at 2 dBm samples were filtered between 620 and 630 mW.

These thresholds were selected based on the power profiles observed in the graphs provided in the assignment document, where distinct clusters correspond to different operating states.

2.2 States Duration Measurements

Using the *micros ()* function in Wokwi, we measured the average duration of each state:

STATES	TIME (average)
Sensor Reading	T_sensorReading ≈ 256,48 us
MsgBuilding	T_msgBuilding ≈ 296,52 us
Transmission at 19.5 dBm	T_trans ≈ 378,87 us
WiFi on	T_wifiOn ≈ 0,2 s
Idle	T_idle ≈ 833,13 us
	$T_{active} \approx 0{,}202\ s$
Deep Sleep	T_deepSleep = 2,3 s
	$T_{cycle} \approx 2{,}502\ s$

The total cycle duration T_{cycle} is obtained as the sum of the active time and the deep sleep time.

Here is an example of the data we gathered:

C	D	E	F	G
T_sensorReading	T_msgBuilding	T_trans	T_wifiOn	T_idle
214	320	443	194831	1023
274	321	438	194894	524
255	320	356	195281	522
256	257	428	195545	990
274	320	494	195202	1023
256	321	356	195147	501
256	329	356	194952	1005
266	329	429	194848	1026
256	273	355	195299	1027
256	273	355	194904	1023
273	329	441	194815	1004
256	255	356	195301	1027
255	255	355	195444	503
255	255	355	195399	1024
256	329	356	195157	504
256	255	355	195145	503
255	328	355	194931	1023
255	255	355	195471	861
255	329	355	195441	505
255	254	355	195315	1004
254	329	355	195166	502
255	255	355	195298	1023
256	329	356	194948	1015
256,4782609	296,5217391	378,8695652	195162,3478	833,1304348

Where the last row reports the average value of each measured time component.

2.2.1 Some Clarifications

1. T_sensorReading represents the time required to acquire data from the sensors.

   This includes reading the motion state from the PIR sensor and the ambient light value from the LDR using *digitalRead ()* and *analogRead ().*
2. T_msgBuilding corresponds to the time needed to construct the message string containing the sensor information.

   This includes formatting the message based on the motion detection result and embedding the luminosity value.
3. T_wifiOn represents the time required to initialize the WiFi interface and set up ESP-NOW communication.

   This includes configuring the device in station mode, initializing ESP-NOW, and registering the send callback, and adding the peer.
4. T_trans represents the duration of the transmission phase.

   It is measured from the call to *esp_now_send ()* until the send callback is triggered, indicating that the data has been successfully sent.
5. T_idle corresponds to the remaining active time during which the system is awake but not performing any specific operation.

   It is computed as the difference between the total awake time and the sum of all measured active states.
6. T_deepSleep represents the deep sleep duration, during which the node enters a low-power state.

2.3 Energy Consumption

The energy consumption in each state is determined by multiplying the power consumption by the duration of that state, according to the relation $E = P \times T$:

STATES	POWER	TIME	ENERGY
Sensor Reading	351,09 mW	0,00026 s	$E_{read} \approx 0{,}09$ mJ
MsgBuilding	351,09 mW	0,0003 s	$E_{msgBuild} \approx 0{,}1$ mJ
Transmission at 19.5 dBm	686,2 mW	0,00038 s	$E_{trans} \approx 0{,}26$ mJ
WiFi on	633,23 mW	0,2 s	$E_{w} \approx 126{,}646$ mJ
Idle	237,05 mW	0,00083 s	$E_{idle} \approx 0{,}2$ mJ
Deep Sleep	45,78 mW	2,3 s	$E_{ds} \approx 105{,}29$ mJ
			$E_{cons} \approx 232{,}586$ mJ

The total energy consumed per transmission cycle (E_{cons}) is calculated by summing the energy contributions of all states.

2.4 Operating Lifetime

Battery Energy:

$E_{battery} = ABCD \,\%\, 5000 + 15000 \quad [10802468 \rightarrow \text{ABCD} = 2468\,]$

$E_{battery} = 2468 \,\%\, 5000 + 15000 = 17468\, J$

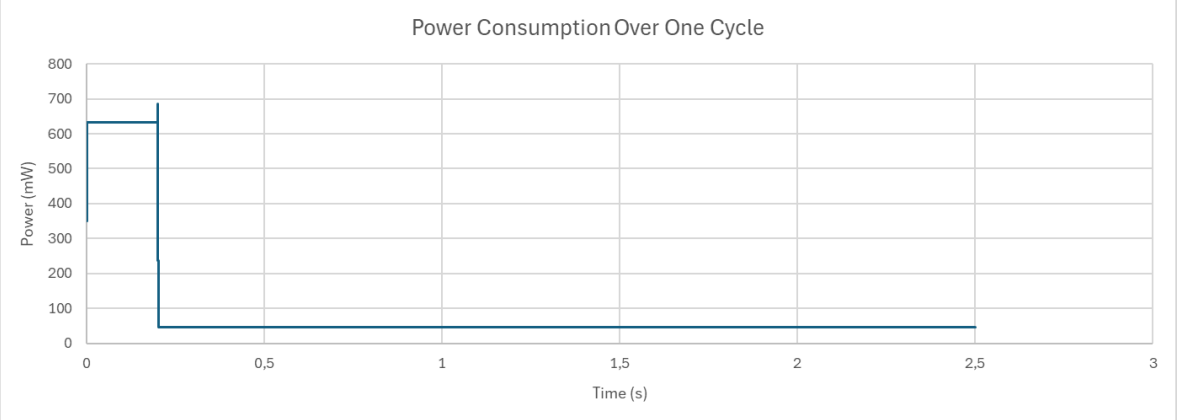
Operating Lifetime:

$$Battery_{life} = \frac{E_{battery}}{E_{consumption}} \times T_{cycle}$$

$$Battery_{life} = \frac{17468\, J}{232{,}586\, mJ} \times 2{,}502\, s \approx 187909\, s \approx 3132\, min \approx 52{,}2\, h \approx 2{,}2\, days$$

2.5 Duty Cycle

The following graph represents an estimation of the duty cycle of the sensor given all of the previous durations and power consumptions of each state:



3| Comment Results and Improvements

From our results, we observe that the battery lifetime is very limited, as it lasts only about two days. This indicates that the current system consumes a relatively high amount of energy per cycle, and therefore, it can be better implemented.

According to the energy model, the node lifetime is directly proportional to the available battery energy and the cycle duration, and inversely proportional to the energy consumed in each cycle. Therefore, we could minimize the energy consumption of the most demanding states, or maximize the cycle duration and/or battery energy.

3.1 Minimize Energy Consumption

3.1.1 Reduce WiFi Initialization and Transmission Procedure

Looking at our duty cycle graph, we can observe that the WiFi ON phase dominates the active time (0.2 s), and combined with its relatively high power consumption, it contributes approximately 54% of the total energy usage.

In our current implementation, the WiFi module is initialized at every wake-up, which introduces a significant overhead. Although the WiFi initialization procedure is already minimal, anyway we can reduce the number of times the WiFi module is activated.

One possible way is to avoid unnecessary transmissions by storing the previous sensor values in RTC memory and then letting the node compare them with the current measurements after waking up. If no significant change is detected then for the sink node it is unnecessary to elaborate the same information for lighting control. For this reason, the node can skip both the WiFi initialization and the transmission phase, and directly return to deep sleep.

Here we improved a new version of implementation:

[IoTChallengeOpt.ino Copy - Wokwi ESP32, STM32, Arduino Simulator]

In this case, two different cycle energies must be considered: one when a transmission occurs (TX cycle) and one when no transmission is performed (No-TX cycle).

TX cycle:

$$E_{TXcycle} = E_{read} + E_{msgBuild} + E_{w} + E_{trans} + E_{idle} + E_{ds} \approx 232,586\,mJ$$

NO-TX cycle (the new values are from the new implementation):

$$T_{newMsgBuild} \approx 20\,us = 0,00002\,s$$

$$E_{newMsgBuild} = 351,09\,mW \times 0,00002\,s \approx 0,007\,mJ$$

$T_{newIdle} \approx 42\,us \approx 0,00004\,s$

$E_{newIdle} = 237,05\,mW \times 0,00004\,s \approx 0,009\,mJ$

$E_{NO-TXcycle} = E_{read} + E_{newMsgBuild} + E_{newIdle} + E_{ds} \approx 105,4\,mJ$

Let P be the probability that a transmission is required and assume P = 0,5.

$E_{avg} = P \times E_{TXcycle} + (1 - P) \times E_{NO-TXcycle} \approx 168,993\,mJ$

$T_{TXcycle} = T_{read} + T_{msgBuild} + T_{w} + T_{trans} + T_{idle} + T_{ds} \approx 2,502\,s$

$T_{NO-TXcycle} = T_{read} + T_{newMsgBuild} + T_{newIdle} + T_{ds} \approx 2,3\,s$

$T_{avg} = P \times T_{TXcycle} + (1 - P) \times T_{NO-TXcycle} \approx 2,401\,s$

Calculating the lifetime:

$Battery_{life} = \frac{17468\,J}{168,993\,mJ} \times 2,401\,s \approx 248179\,s \approx 4136\,min \approx 68\,h \approx 2,87\,days.$

Now assuming P = 0,3.

$E_{avg} = P \times E_{TXcycle} + (1 - P) \times E_{NO-TXcycle} \approx 143,556\,mJ$

$T_{avg} = P \times T_{TXcycle} + (1 - P) \times T_{NO-TXcycle} \approx 2,36\,s$

Calculating the lifetime:

$Battery_{life} = \frac{17468\,J}{143,556\,mJ} \times 2,36\,s \approx 287166\,s \approx 4786\,min \approx 79\,h \approx 3,3\,days.$

Now assuming P = 0,1.

$E_{avg} = P \times E_{TXcycle} + (1 - P) \times E_{NO-TXcycle} \approx 118,119\,mJ$

$T_{avg} = P \times T_{TXcycle} + (1 - P) \times T_{NO-TXcycle} \approx 2,32\,s$

Calculating the lifetime:

$Battery_{life} = \frac{17468\,J}{118,119\,mJ} \times 2,32\,s \approx 343092\,s \approx 5718\,min \approx 95\,h \approx 4\,days.$

This demonstrates that reducing the number of WiFi activations can greatly improve the battery lifetime.

3.1.2 Change Transmission Power from 19,5 dBm to 2 dBm

Another possible way is to reduce the transmission power from 19.5 dBm to 2 dBm.

In this case, the average transmission power value decreased from 686,2 mW to 623,62 mW. As a result, the energy consumed during the transmission phase is reduced from 0,26 mJ to 0,24 mJ, and the total energy consumption becomes 232,566 mJ. Computing

the lifetime using the new value:

$Battery_{life} = \frac{17468\,J}{232{,}566\,mJ} \times 2{,}502\,s \approx 187925\,s \approx 3133\,min \approx 52{,}2\,h \approx 2{,}2\,days$

Although this improvement slightly reduces the energy consumption, its overall impact on the total energy per cycle is limited, and the resulting increase in battery lifetime is negligible.

3.2 Maximize Battery Energy and Cycle Duration

Another possible improvement is to increase the deep sleep duration, and therefore, the cycle duration would increase. Same for battery energy.

If we consider a person code ending with 4999:

$X = \frac{(AB\ \%\ 50)+5}{10} = \frac{(99\ \%\ 50)+5}{10} = 5{,}4\,s$

$E_{battery} = ABCD\ \%\ 5000 + 15000 = 4999\ \%\ 5000 + 15000 = 19999\,J$

$E_{newDs} \approx$ 247,212 mJ

$E_{newCons} \approx$ 374,508 mJ

Recalculating the T_cycle using $T_{active} \approx 0{,}202\,s$, we obtain about 5,602 s.

Then, $Battery_{life} = \frac{19999\,J}{374{,}508\,mJ} \times 5{,}602\,s \approx 299150\,s \approx 4985\,min \approx 83\,h \approx 3{,}5\,days$

We can observe that although increasing the deep sleep duration extends the cycle time and can improve the battery lifetime, it also increases the energy consumed per cycle, as the energy contribution of the deep sleep state grows with its duration. Therefore, this approach involves a trade-off between longer cycles and higher per-cycle energy consumption.